

# TaxTrack

## Software Requirements Specification

Project Title: TaxTrack — Transparent Tax Tracking System

| Document Type    | Software Requirements Specification (SRS)     |
|------------------|-----------------------------------------------|
| Project Name     | TaxTrack — Transparent Tax Tracking System    |
| Version          | 1.0                                           |
| Technology Stack | MERN (MongoDB, Express.js, React.js, Node.js) |
| Portals          | Taxpayer Portal + Government Portal           |
| Date             | June 2025                                     |

# 1. Introduction

## 1.1 Purpose

The purpose of this document is to define the software requirements for TaxTrack — a MERN-stack web application that enables citizens to pay taxes online, receive unique token codes for each transaction, and transparently track how their tax money flows into government pools and is allocated to public projects.

The system features two portals: a **Taxpayer Portal** for citizens and a **Government Portal** for authorized government officials. This SRS document serves as the foundation for design, development, testing, and deployment of the system.

## 1.2 Scope

TaxTrack is a full-stack web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js). The system enables:

- Taxpayers to register, login, and pay various types of taxes (Income Tax, GST, Property Tax, Corporate Tax, Customs Duty)
- Automatic generation of a unique token code (e.g., TXN-XXXXXXXX) upon successful payment
- Automatic routing of payments to appropriate tax pools based on tax type
- Government officials to view total pool balances, allocate funds to named public projects, and manage multiple pools
- Taxpayers to track the complete flow of their payment — from payment → pool → project allocation

## 1.3 Overview

This document describes the functional requirements, non-functional requirements, user interface requirements, system interfaces, performance requirements, and security requirements of the TaxTrack system. It includes entity-relationship information, data flow descriptions, class structures, and state transitions relevant to the system.

## 2. General Description

### 2.1 System Functions

The main functions of the TaxTrack system include:

| Function                  | Description                                                                                           | Portal     |
|---------------------------|-------------------------------------------------------------------------------------------------------|------------|
| User Registration & Login | Taxpayers register with name, email, PAN, and password. Government officials log in with official ID. | Both       |
| Tax Payment               | Taxpayers select tax type, enter amount, and submit payment. Tax receipt is generated instantly.      | Taxpayer   |
| Token Generation          | A unique cryptographic-style token (TXN-XXXXXXXX) is issued upon payment for tracking.                | Taxpayer   |
| Pool Auto-Routing         | Payments auto-route to the correct tax pool based on tax type.                                        | System     |
| Fund Tracking             | Taxpayers see full fund flow: payment → pool → project allocation.                                    | Taxpayer   |
| Pool Management           | Government creates and views tax pool balances and allocations.                                       | Government |
| Fund Allocation           | Government allocates funds from a pool to a named project with department details.                    | Government |
| Allocation History        | Complete allocation history per pool visible to both taxpayers and government.                        | Both       |
| Analytics Dashboard       | Government sees total collected, allocated, and unallocated funds.                                    | Government |

### 2.2 User Community

#### Taxpayers (Citizens)

Primary users who pay taxes and track their money.

- Register and log in with PAN
- Pay taxes (Income Tax, GST, Property Tax, etc.)
- Receive unique token per payment
- View payment history with token codes
- Track fund flow: payment → pool → project allocation

#### Government Officials

Authorized government personnel managing public funds.

- Log in with official ID and secure password
- View all tax payments received
- Create and manage multiple tax pools
- Allocate pool funds to public projects
- Monitor fund allocation with department tracking

### 3. Functional Requirements

#### 3.1 Possible Outcomes

| Outcome               | Description                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Tax Payment Submitted | Taxpayer receives a unique token code and payment is confirmed in the system.                                 |
| Pool Balance Updated  | The relevant tax pool balance increases by the payment amount.                                                |
| Fund Allocated        | Pool balance decreases and a project allocation record is created with full details.                          |
| Tracking Displayed    | Taxpayer sees the complete flow: their payment, which pool it entered, and which projects received the funds. |
| Auth Success/Failure  | Valid credentials grant portal access; invalid credentials show an error.                                     |

#### 3.2 Priority-Ranked Requirements

| Rank | Feature                           | Description                                            |
|------|-----------------------------------|--------------------------------------------------------|
| 1    | Tax Payment with Token Generation | Core taxpayer transaction with unique token issuance   |
| 2    | Fund Pool Management              | Government pool creation and balance tracking          |
| 3    | Fund Allocation to Projects       | Government allocates from pool to public projects      |
| 4    | Payment Flow Tracking             | Taxpayer sees money flow from payment → pool → project |
| 5    | User Authentication (JWT)         | Secure role-based login for both portals               |
| 6    | Tax Pool Overview                 | Taxpayers see all pools and their balances             |
| 7    | Audit Trail                       | Complete history of all payments and allocations       |

#### 3.3 Input–Output Relationships

| Input                              | Output                                                                        |
|------------------------------------|-------------------------------------------------------------------------------|
| Taxpayer submits tax amount + type | Unique token code (TXN-XXXXXXXX) + confirmation + pool entry                  |
| Government allocates from pool     | Pool balance decreases, project record created, allocation timestamp          |
| Taxpayer views track page          | Full fund flow: payment details → pool name/balance → all project allocations |
| Taxpayer login credentials         | JWT token + access to taxpayer dashboard                                      |
| Government official ID + password  | JWT token + access to government dashboard                                    |
| Government creates pool            | New pool with zero balance, available for tax routing                         |

#### 3.4 Detailed Functional Requirements

## **User Authentication**

- The system shall allow taxpayers to register with name, email, PAN number, and password.
- The system shall authenticate users using email and password via JWT.
- The system shall provide role-based access (taxpayer vs government).
- The system shall maintain secure sessions using JSON Web Tokens (JWT) with 7-day expiry.
- The system shall prevent unauthorized access to protected API routes.

## **Tax Payment (Taxpayer)**

- The system shall allow taxpayers to select tax type from: Income Tax, GST, Property Tax, Corporate Tax, Customs Duty.
- The system shall allow taxpayers to enter any positive amount for payment.
- The system shall generate a unique token code in format TXN-XXXXXXX upon successful payment.
- The system shall auto-route the payment to the correct tax pool based on tax type.
- The system shall display a real-time payment success screen with token and pool info.

## **Payment Tracking (Taxpayer)**

- The system shall display all past payments with token codes, amounts, and dates.
- The system shall allow token-based search of payment history.
- The system shall display the full fund flow for each selected payment.
- The system shall show pool details including current balance and all project allocations.
- The system shall display each project allocation from the pool in chronological order.

## **Pool Management (Government)**

- The system shall allow government officials to create new named tax pools.
- The system shall display current balance and total collected for each pool.
- The system shall show allocation percentage as a visual progress bar.
- The system shall list all allocations per pool with project name, department, and amount.

## **Fund Allocation (Government)**

- The system shall allow government to allocate funds from a pool to a named project.
- The system shall require project name, amount, and department for each allocation.
- The system shall prevent allocation if amount exceeds available pool balance.
- The system shall record allocation timestamp and allocating official.
- The system shall update pool balance immediately upon successful allocation.

## 4. User Interface Requirements

### 4.1 Software Interfaces

| Interface          | Technology                    | Purpose                           |
|--------------------|-------------------------------|-----------------------------------|
| Web Browser        | Chrome, Firefox, Edge, Safari | Primary client access point       |
| Frontend Framework | React.js 18 + Vite            | Single-page application rendering |
| Backend API        | Node.js + Express.js          | RESTful API serving JSON data     |
| Database           | MongoDB + Mongoose ODM        | Document-based data storage       |
| Authentication     | JWT (jsonwebtoken)            | Stateless auth tokens             |
| Password Security  | bcryptjs                      | Password hashing and comparison   |
| CORS               | cors middleware               | Cross-origin request handling     |

### 4.2 Portal Interface Examples

#### Taxpayer Portal Pages

- Landing Page — Portal selection with Taxpayer and Government entry points
- Login/Register Page — Toggle between login and registration with PAN input
- Dashboard — Stats cards (total paid, payments count, active pools), recent payments, pool overview, fund flow explainer
- Pay Tax Page — Tax type selector, amount input, live payment preview, success screen with token card
- Track Payment Page — Payment list with token search, detailed fund flow panel per payment

#### Government Portal Pages

- Login Page — Official ID and secure password with auth indicator
- Dashboard — 4 stat cards (total collected, in pools, allocated, taxpayer count), tabbed interface
- Tax Pools Tab — Pool cards with balance, progress bar, allocation list, and "Allocate" button
- All Payments Tab — Table of all payments with token, taxpayer, amount, type, pool, date
- Allocations Tab — Master list of all project allocations across all pools
- Create Pool Modal — Form to create new tax pool with name, category, description
- Allocate Funds Modal — Project name, amount, department selector, description

## 5. Performance Requirements

### 5.1 Response Time

The system should respond to user actions within the following timeframes:

| Operation              | Target Time   | Notes                                     |
|------------------------|---------------|-------------------------------------------|
| Page load / navigation | < 2 seconds   | React SPA client-side routing             |
| User login / JWT issue | < 1 second    | Database lookup + token generation        |
| Tax payment processing | < 2 seconds   | DB write + token generation + pool update |
| Payment history fetch  | < 1.5 seconds | MongoDB indexed query                     |
| Pool balance update    | < 1 second    | Atomic document update                    |
| Fund allocation        | < 2 seconds   | Pool document update + subdocument push   |
| Dashboard analytics    | < 3 seconds   | Aggregation across collections            |

### 5.2 Throughput and Scalability

- The system shall handle at least 500 concurrent taxpayer sessions.
- MongoDB horizontal scaling via sharding supports future data growth.
- JWT stateless auth eliminates server-side session storage bottlenecks.
- React SPA architecture reduces server load via client-side rendering.
- Additional tax pools can be created without schema changes.

## 6. Non-Functional Attributes

### Usability

- Both portals shall have a clear, color-coded design (Gold = Taxpayer, Green = Government).
- Tax type selection uses visual button toggles, not dropdowns.
- Payment success page uses large token display for easy reading and copying.
- Fund flow tracking uses a step-by-step visual flow (arrows and icons).
- Government dashboard uses tabbed layout to separate concerns.

### Security

- All passwords stored using bcryptjs with salt rounds of 10.
- JWT tokens expire after 7 days; stored in localStorage.
- API routes protected by auth middleware checking token validity.

- Government-only routes additionally check `user.role === "government"`.
- Token codes generated using cryptographically random alphanumeric characters.
- CORS configured to allow only specified frontend origins.

## **Reliability**

- MongoDB ACID guarantees at the document level ensure payment atomicity.
- Pool balance and `totalCollected` are updated in the same Mongoose `save()` call.
- Token uniqueness enforced via MongoDB unique index with retry loop.
- Allocation amount validated server-side against live pool balance.

## **Maintainability**

- Modular codebase: separate models, routes, and middleware.
- RESTful API design makes frontend/backend independently replaceable.
- Mongoose schemas with validation centralize data integrity rules.
- Environment variables via `.env` file for easy configuration changes.



## 7. Entity-Relationship Design

### 7.1 Entities and Attributes

#### Entity: USER

| Attribute  | Data Type                   | Constraint                     |
|------------|-----------------------------|--------------------------------|
| _id        | ObjectId                    | Primary Key                    |
| name       | String                      | NOT NULL                       |
| email      | String                      | UNIQUE, NOT NULL               |
| password   | String (hashed)             | NOT NULL                       |
| pan        | String                      | UNIQUE, sparse (taxpayer only) |
| role       | Enum: taxpayer / government | DEFAULT: taxpayer              |
| officialId | String                      | Government officials only      |
| createdAt  | DateTime                    | Auto-generated                 |

#### Entity: PAYMENT

| Attribute    | Data Type          | Constraint                                                  |
|--------------|--------------------|-------------------------------------------------------------|
| _id          | ObjectId           | Primary Key                                                 |
| taxpayer     | ObjectId → USER    | Foreign Key                                                 |
| taxpayerName | String             | Denormalized for query speed                                |
| amount       | Number             | NOT NULL, must be > 0                                       |
| taxType      | Enum               | income_tax / gst / property_tax / corporate_tax / customs_d |
| description  | String             | Optional                                                    |
| tokenCode    | String             | UNIQUE, TXN-XXXXXXXX format                                 |
| pool         | ObjectId → TAXPOOL | Foreign Key                                                 |
| status       | Enum               | pending / confirmed / failed                                |
| createdAt    | DateTime           | Auto-generated                                              |

#### Entity: TAXPOOL

| Attribute | Data Type | Constraint  |
|-----------|-----------|-------------|
| _id       | ObjectId  | Primary Key |

|                |                            |                                |
|----------------|----------------------------|--------------------------------|
| name           | String                     | Pool display name              |
| taxCategory    | String                     | Tax type served by this pool   |
| description    | String                     | Optional                       |
| balance        | Number                     | Current unallocated balance    |
| totalCollected | Number                     | Cumulative sum of all payments |
| allocations    | Array of ALLOCATION subdoc | Embedded subdocuments          |
| createdBy      | ObjectId → USER            | Government official FK         |
| createdAt      | DateTime                   | Auto-generated                 |

**Entity: ALLOCATION (Subdocument of TAXPOOL)**

| Attribute   | Data Type       | Constraint                   |
|-------------|-----------------|------------------------------|
| projectName | String          | Name of the funded project   |
| amount      | Number          | Amount allocated             |
| department  | String          | Ministry/department          |
| description | String          | Optional project description |
| allocatedBy | ObjectId → USER | Foreign Key to Gov official  |
| allocatedAt | DateTime        | Default: Date.now            |

**7.2 Relationships**

| Relationship | Entity 1   | Entity 2   | Cardinality | Notes                            |
|--------------|------------|------------|-------------|----------------------------------|
| Pays         | USER       | PAYMENT    | 1 → Many    | One taxpayer makes many payments |
| Routes To    | PAYMENT    | TAXPOOL    | Many → 1    | Many payments enter one pool     |
| Contains     | TAXPOOL    | ALLOCATION | 1 → Many    | Embedded subdocuments            |
| Manages      | USER (Gov) | TAXPOOL    | 1 → Many    | Government creates pools         |
| Authorizes   | USER (Gov) | ALLOCATION | 1 → Many    | Official allocates funds         |

## 8. API Reference

### Authentication Routes

Base: */api/auth*

| Method | Endpoint           | Auth | Description                                           |
|--------|--------------------|------|-------------------------------------------------------|
| POST   | /taxpayer/register | None | Register new taxpayer with name, email, PAN, password |
| POST   | /taxpayer/login    | None | Taxpayer login — returns JWT + user object            |
| POST   | /gov/login         | None | Government login with officialId + password           |

### Payment Routes

Base: */api/payments*

| Method | Endpoint | Auth           | Description                                        |
|--------|----------|----------------|----------------------------------------------------|
| POST   | /        | Taxpayer JWT   | Create tax payment, returns payment + token + pool |
| GET    | /my      | Taxpayer JWT   | Get authenticated taxpayer's payment history       |
| GET    | /all     | Government JWT | Get all payments across all taxpayers              |

### Pool Routes

Base: */api/pools*

| Method | Endpoint      | Auth           | Description                                     |
|--------|---------------|----------------|-------------------------------------------------|
| GET    | /             | Any JWT        | Get all tax pools with balances and allocations |
| POST   | /             | Government JWT | Create a new tax pool                           |
| POST   | /:id/allocate | Government JWT | Allocate funds from pool to a project           |

## 9. Class Diagram

The TaxTrack system is modeled using the following classes:

| Class                    | Attributes                                                                                                                                                                                                                                                                 | Methods                                                                                                                                                     |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User                     | <ul style="list-style-type: none"><li>• userId: ObjectId</li><li>• name: String</li><li>• email: String</li><li>• password: String (hashed)</li><li>• pan: String</li><li>• role: Enum</li><li>• officialId: String</li></ul>                                              | <ul style="list-style-type: none"><li>• register()</li><li>• login()</li><li>• logout()</li><li>• comparePassword()</li><li>• getProfile()</li></ul>        |
| Payment                  | <ul style="list-style-type: none"><li>• paymentId: ObjectId</li><li>• taxpayer: Ref&lt;User&gt;</li><li>• amount: Number</li><li>• taxType: Enum</li><li>• tokenCode: String</li><li>• pool: Ref&lt;TaxPool&gt;</li><li>• status: Enum</li><li>• createdAt: Date</li></ul> | <ul style="list-style-type: none"><li>• createPayment()</li><li>• generateToken()</li><li>• getByTaxpayer()</li><li>• getAll()</li></ul>                    |
| TaxPool                  | <ul style="list-style-type: none"><li>• poolId: ObjectId</li><li>• name: String</li><li>• taxCategory: String</li><li>• balance: Number</li><li>• totalCollected: Number</li><li>• allocations: Allocation[]</li><li>• createdBy: Ref&lt;User&gt;</li></ul>                | <ul style="list-style-type: none"><li>• create()</li><li>• addFunds(amount)</li><li>• allocate(project)</li><li>• getAll()</li><li>• getBalance()</li></ul> |
| Allocation (Subdocument) | <ul style="list-style-type: none"><li>• projectName: String</li><li>• amount: Number</li><li>• department: String</li><li>• description: String</li><li>• allocatedBy: Ref&lt;User&gt;</li><li>• allocatedAt: Date</li></ul>                                               | <ul style="list-style-type: none"><li>• create()</li><li>• getSummary()</li></ul>                                                                           |

### Class Relationships

| Relationship Type | Between              | Description                            |
|-------------------|----------------------|----------------------------------------|
| Association       | User → Payment       | Taxpayer makes payments (1 to many)    |
| Association       | Payment → TaxPool    | Payment routes to pool (many to 1)     |
| Composition       | TaxPool → Allocation | Pool owns allocations as embedded docs |
| Dependency        | TaxPool ← User (Gov) | Government manages pool lifecycle      |
| Aggregation       | TaxPool ← Payment[]  | Pool aggregates multiple payments      |

## 10. System Flowchart

### 10.1 Taxpayer Flow

| Step | Activity                       | Decision/Output                                    |
|------|--------------------------------|----------------------------------------------------|
| 1    | User visits TaxTrack           | Sees portal selection (Taxpayer / Government)      |
| 2    | Selects Taxpayer Portal        | Navigates to /taxpayer/login                       |
| 3    | Already registered?            | YES → Login   NO → Register                        |
| 4    | Register (if new)              | Enter name, email, PAN, password → Account created |
| 5    | Login                          | Enter email + password → JWT issued                |
| 6    | Credentials valid?             | YES → Dashboard   NO → Error message               |
| 7    | View Dashboard                 | See total paid, payments count, pool overview      |
| 8    | Click "Pay Tax"                | Navigates to payment form                          |
| 9    | Select tax type + enter amount | Live preview shown                                 |
| 10   | Submit payment                 | Backend creates payment, routes to pool            |
| 11   | Receive token                  | TXN-XXXXXXX displayed on success screen            |
| 12   | Click "Track Money"            | Navigates to /taxpayer/track                       |
| 13   | Select payment from list       | Full fund flow: payment → pool → projects shown    |
| 14   | Logout / Exit                  | Session cleared                                    |

### 10.2 Government Flow

| Step | Activity                     | Decision/Output                                             |
|------|------------------------------|-------------------------------------------------------------|
| 1    | Select Government Portal     | Navigates to /gov/login                                     |
| 2    | Enter Official ID + Password | Credentials verified                                        |
| 3    | Valid credentials?           | YES → Government Dashboard   NO → Error                     |
| 4    | View Dashboard               | Stats: Total collected, in pools, allocated, taxpayer count |
| 5    | View Tax Pools Tab           | All pools with balance, progress bar, allocations           |
| 6    | Pool has balance?            | YES → Allocate button active   NO → Pool Empty              |
| 7    | Click Allocate               | Modal opens: enter project name, amount, department         |
| 8    | Amount ≤ pool balance?       | YES → Allocate confirmed   NO → Error                       |
| 9    | View All Payments            | Table of all taxpayer payments across system                |

|    |                      |                                                |
|----|----------------------|------------------------------------------------|
| 10 | View Allocations Tab | Master list of all project allocations         |
| 11 | Create New Pool      | Name, category, description → New pool created |

# 11. Data Flow Diagram (DFD)

## 11.1 External Entities

| Entity              | Role         | Inputs to System                                                   | Outputs from System                               |
|---------------------|--------------|--------------------------------------------------------------------|---------------------------------------------------|
| Taxpayer            | Primary user | Registration data, login credentials, payment amount, payment type | JWT token, payment token code, fund tracking data |
| Government Official | Admin user   | Official credentials, pool data, allocation request                | JWT token, pool analytics, all payment data       |

## 11.2 Data Stores

| ID | Data Store    | Contents                                                      | Technology             |
|----|---------------|---------------------------------------------------------------|------------------------|
| D1 | User Store    | Taxpayer & government accounts, hashed passwords, PANs        | Official DB Collection |
| D2 | Payment Store | All tax payments, token codes, amounts, types, pool reference | MongoDB Collection     |
| D3 | Pool Store    | Tax pools, balances, totalCollected, allocation sub-documents | MongoDB Collection     |

## 11.3 Level 0 DFD — Context Diagram

The entire TaxTrack system is represented as a single process receiving inputs from Taxpayer and Government entities and returning appropriate outputs.

| From Entity | Data Flow to System     | System Process  | Data Flow from System            | To Entity  |
|-------------|-------------------------|-----------------|----------------------------------|------------|
| Taxpayer    | Registration/login data | TaxTrack System | JWT + dashboard access           | Taxpayer   |
| Taxpayer    | Payment amount + type   | TaxTrack System | Token code + pool info           | Taxpayer   |
| Taxpayer    | Track request           | TaxTrack System | Fund flow details                | Taxpayer   |
| Government  | Official credentials    | TaxTrack System | JWT + dashboard access           | Government |
| Government  | Allocation request      | TaxTrack System | Updated pool + allocation record | Government |

## 11.4 Level 1 DFD — Process Decomposition

| Process ID | Process Name    | Inputs                         | Outputs                               | Data Stores Used          |
|------------|-----------------|--------------------------------|---------------------------------------|---------------------------|
| P1.0       | User Auth       | Credentials, registration data | JWT token, user object                | D1 – Users                |
| P2.0       | Tax Payment     | Amount, tax type, user JWT     | Token code, payment record, pool info | D2 – Payments, D3 – Pools |
| P3.0       | Pool Routing    | Tax type                       | Selected pool ID                      | D3 – Pools                |
| P4.0       | Fund Tracking   | User JWT                       | Payment history + pool + allocation   | D2, D3                    |
| P5.0       | Pool Management | Gov JWT, pool data             | Pool created/updated                  | D3 – Pools                |

|      |                 |                                                           |                                   |
|------|-----------------|-----------------------------------------------------------|-----------------------------------|
| P6.0 | Fund Allocation | Gov JWT, project details, allocation record, pool balance | DC - updates                      |
| P7.0 | Analytics       | Gov JWT                                                   | Aggregated financial stats D2, D3 |



## 12. State Diagrams

### 12.1 Payment State Transitions

| Current State | Event                       | Next State | Action                                         |
|---------------|-----------------------------|------------|------------------------------------------------|
| (Initial)     | Taxpayer submits payment    | Pending    | Record created, pool routing begins            |
| Pending       | Payment DB write successful | Confirmed  | Token generated, pool balance updated          |
| Pending       | DB write fails / error      | Failed     | Error returned to taxpayer                     |
| Confirmed     | Taxpayer views tracking     | Confirmed  | Fund flow displayed (no state change)          |
| Confirmed     | (Final state)               | —          | Payment remains in confirmed state permanently |

### 12.2 Pool State Transitions

| Current State | Event                      | Next State         | Action                                      |
|---------------|----------------------------|--------------------|---------------------------------------------|
| (Initial)     | Government creates pool    | Empty (balance=0)  | Pool record created                         |
| Empty         | Tax payment routed to pool | Active (balance>0) | balance += amount, totalCollected += amount |
| Active        | More payments routed       | Active             | balance increases further                   |
| Active        | Government allocates funds | Active (or Empty)  | balance -= allocation amount                |
| Active        | All funds allocated        | Empty (balance=0)  | Pool ready for new payments                 |

### 12.3 Sequence Diagram — Tax Payment Flow

| Step | Actor              | Message                              | Recipient                  | Response               |
|------|--------------------|--------------------------------------|----------------------------|------------------------|
| 1    | Taxpayer           | POST /api/payments (amount, taxType) | React Client → Express API |                        |
| 2    | Auth Middleware    | Verify JWT token                     | JWT Library                | User object decoded    |
| 3    | Payment Controller | assignPool(taxType)                  | MongoDB                    | Pool document returned |
| 4    | Payment Controller | generateToken()                      | Internal                   | Unique TXN-XXXXXXXX    |
| 5    | Payment Controller | Payment.create(...)                  | MongoDB                    | Payment document saved |
| 6    | Payment Controller | pool.balance += amount; pool.save()  | MongoDB                    | Pool updated           |
| 7    | Express API        | Return {payment, pool}               | React Client               | —                      |
| 8    | React Client       | Display success screen with token    | Taxpayer                   | —                      |

## 13. Decision Table

### 13.1 Tax Payment Decision Table

| Condition / Action     | Rule 1 | Rule 2 | Rule 3 | Rule 4    | Rule 5      |
|------------------------|--------|--------|--------|-----------|-------------|
| User Authenticated?    | Yes    | Yes    | Yes    | No        | Yes         |
| Amount > 0?            | Yes    | No     | Yes    | —         | Yes         |
| Tax Pool Available?    | Yes    | —      | No     | —         | Yes         |
| Token Unique?          | Yes    | —      | —      | —         | No (retry)  |
| — Actions —            |        |        |        |           |             |
| Create Payment Record  | Yes    | No     | No     | No        | Yes (retry) |
| Generate Token         | Yes    | No     | No     | No        | Yes (new)   |
| Update Pool Balance    | Yes    | No     | No     | No        | Yes         |
| Return Success + Token | Yes    | No     | No     | No        | Yes         |
| Return Error Message   | No     | Yes    | Yes    | Yes (401) | No          |

### 13.2 Fund Allocation Decision Table

| Condition / Action          | Rule 1 | Rule 2 | Rule 3 | Rule 4 |
|-----------------------------|--------|--------|--------|--------|
| Official Authenticated?     | Yes    | Yes    | No     | Yes    |
| Pool Exists?                | Yes    | Yes    | —      | No     |
| Amount $\leq$ Pool Balance? | Yes    | No     | —      | —      |
| — Actions —                 |        |        |        |        |
| Deduct from Pool Balance    | Yes    | No     | No     | No     |
| Create Allocation Record    | Yes    | No     | No     | No     |
| Return Updated Pool         | Yes    | No     | No     | No     |
| Return Error (insufficient) | No     | Yes    | No     | No     |
| Return 401 Unauthorized     | No     | No     | Yes    | No     |
| Return 404 Pool Not Found   | No     | No     | No     | Yes    |

## 14. Schedule and Budget

### 14.1 Development Timeline

| Phase                 | Duration  | Key Deliverables                                      |
|-----------------------|-----------|-------------------------------------------------------|
| Requirement Analysis  | 1 Week    | SRS Document, stakeholder review                      |
| System Design         | 1 Week    | ER Diagram, API Design, UI Wireframes                 |
| Backend Development   | 1.5 Weeks | Express API, MongoDB models, JWT auth, pool routing   |
| Frontend Development  | 1.5 Weeks | React pages, both portals, fund flow visualization    |
| Integration & Testing | 1 Week    | API integration, functional testing, security testing |
| Documentation         | 0.5 Weeks | SRS, Lab Document, README                             |
| Deployment (Optional) | 0.5 Weeks | Heroku/Vercel/MongoDB Atlas setup                     |
| Total                 | 7 Weeks   |                                                       |

### 14.2 Cost Estimate

| Item               | Tool/Service                        | Cost                              |
|--------------------|-------------------------------------|-----------------------------------|
| Development Tools  | VS Code, Postman                    | Free                              |
| Frontend Framework | React.js + Vite                     | Free (Open Source)                |
| Backend Framework  | Node.js + Express.js                | Free (Open Source)                |
| Database           | MongoDB Community / Atlas Free Tier | Free                              |
| Authentication     | jsonwebtoken, bcryptjs              | Free (Open Source)                |
| Hosting — Frontend | Vercel (Free tier)                  | \$0                               |
| Hosting — Backend  | Render (Free tier)                  | \$0                               |
| Database Hosting   | MongoDB Atlas M0 (Free)             | \$0                               |
| Estimated Total    |                                     | \$0 – \$20 (if paid hosting used) |

## 15. Appendices

### 15.1 Token Code Format

Each tax payment generates a unique token in the format: **TXN-XXXXXXXX**

Where X is a random character from: A-Z (excluding I, O) and 2-9 (to avoid visual confusion).

Example tokens: TXN-AJ7KP3MN, TXN-B5QRXZ2H, TXN-CN4MWVYA

### 15.2 Tax Type to Pool Mapping

| Tax Type               | Auto-Assigned Pool              | Rationale                                    |
|------------------------|---------------------------------|----------------------------------------------|
| Income Tax             | Infrastructure Development Fund | Primary source for road/rail/bridge projects |
| Property Tax           | Infrastructure Development Fund | Urban infrastructure linkage                 |
| GST (Goods & Services) | Healthcare & Welfare Pool       | Consumer-linked welfare programs             |
| Corporate Tax          | Education & Research Fund       | Business-education linkage                   |
| Customs Duty           | Infrastructure Development Fund | Trade infrastructure support                 |

### 15.3 Glossary

| Term              | Definition                                                                     |
|-------------------|--------------------------------------------------------------------------------|
| Token Code        | Unique alphanumeric identifier (TXN-XXXXXXXX) issued for each tax payment      |
| Tax Pool          | Accumulated fund from tax payments, categorized by tax type or purpose         |
| Allocation        | Transfer of funds from a pool to a named public project                        |
| JWT               | JSON Web Token — stateless authentication mechanism                            |
| PAN               | Permanent Account Number — Indian taxpayer identification                      |
| MERN              | MongoDB, Express.js, React.js, Node.js — the technology stack used             |
| ODM               | Object Document Mapper — Mongoose's MongoDB interface layer                    |
| Subdocument       | Embedded MongoDB document within a parent document (allocations within a pool) |
| Role-Based Access | System restricts API access based on user.role (taxpayer vs government)        |